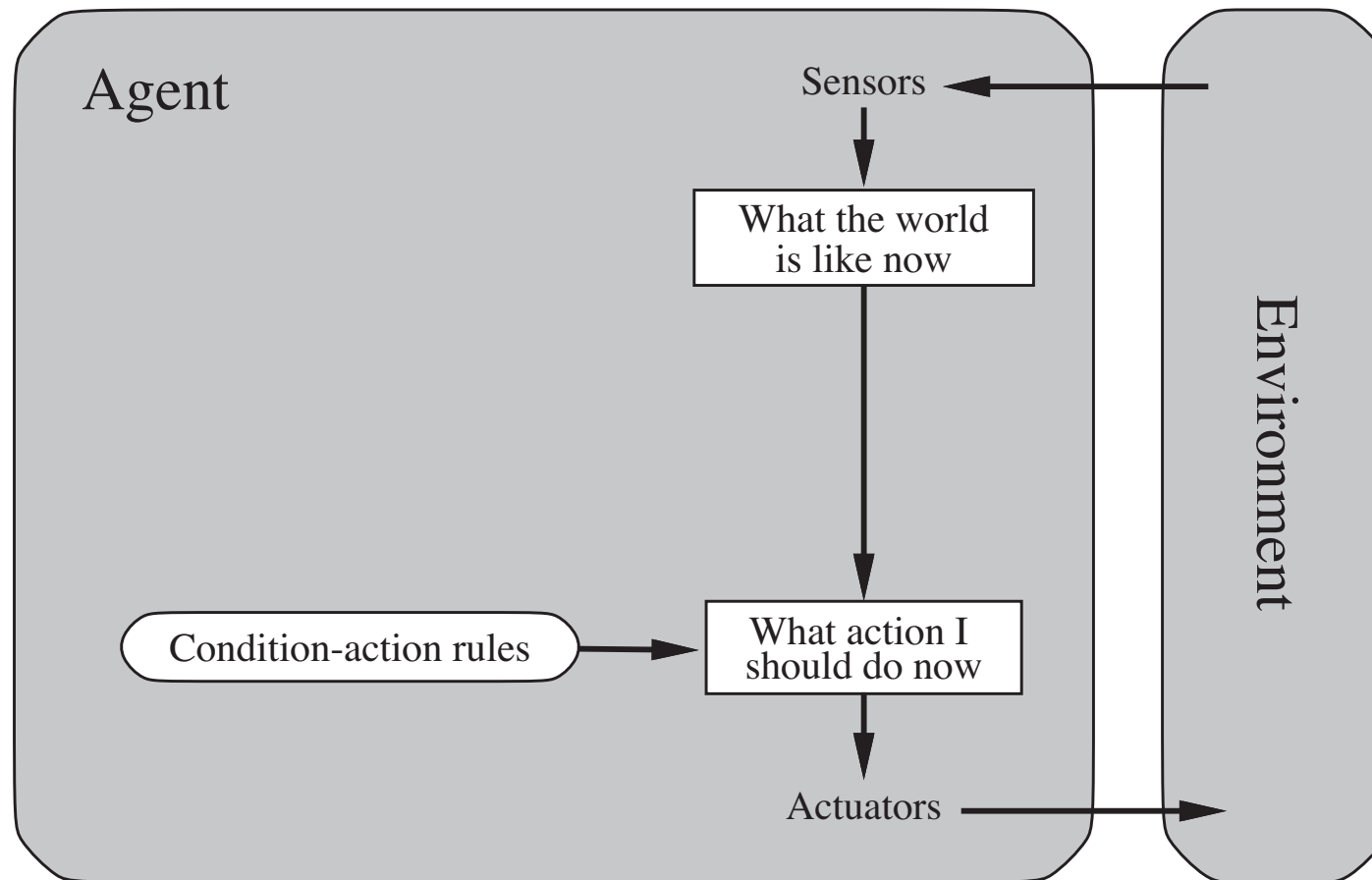


Ευφυείς Πράκτορες και Πολυπρακτορικά Συστήματα

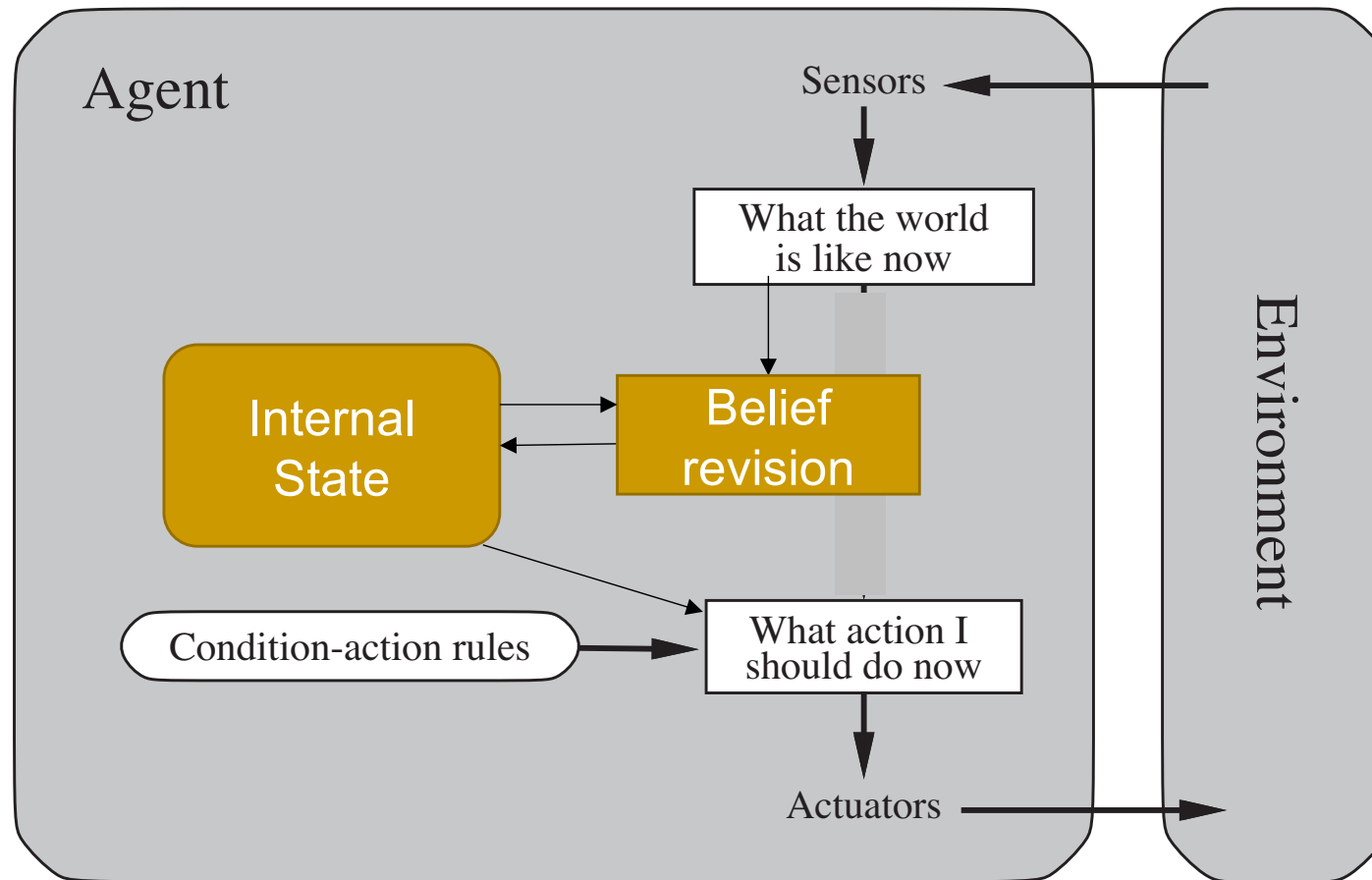
Γεώργιος Βούρος

AGENT ARCHITECTURES

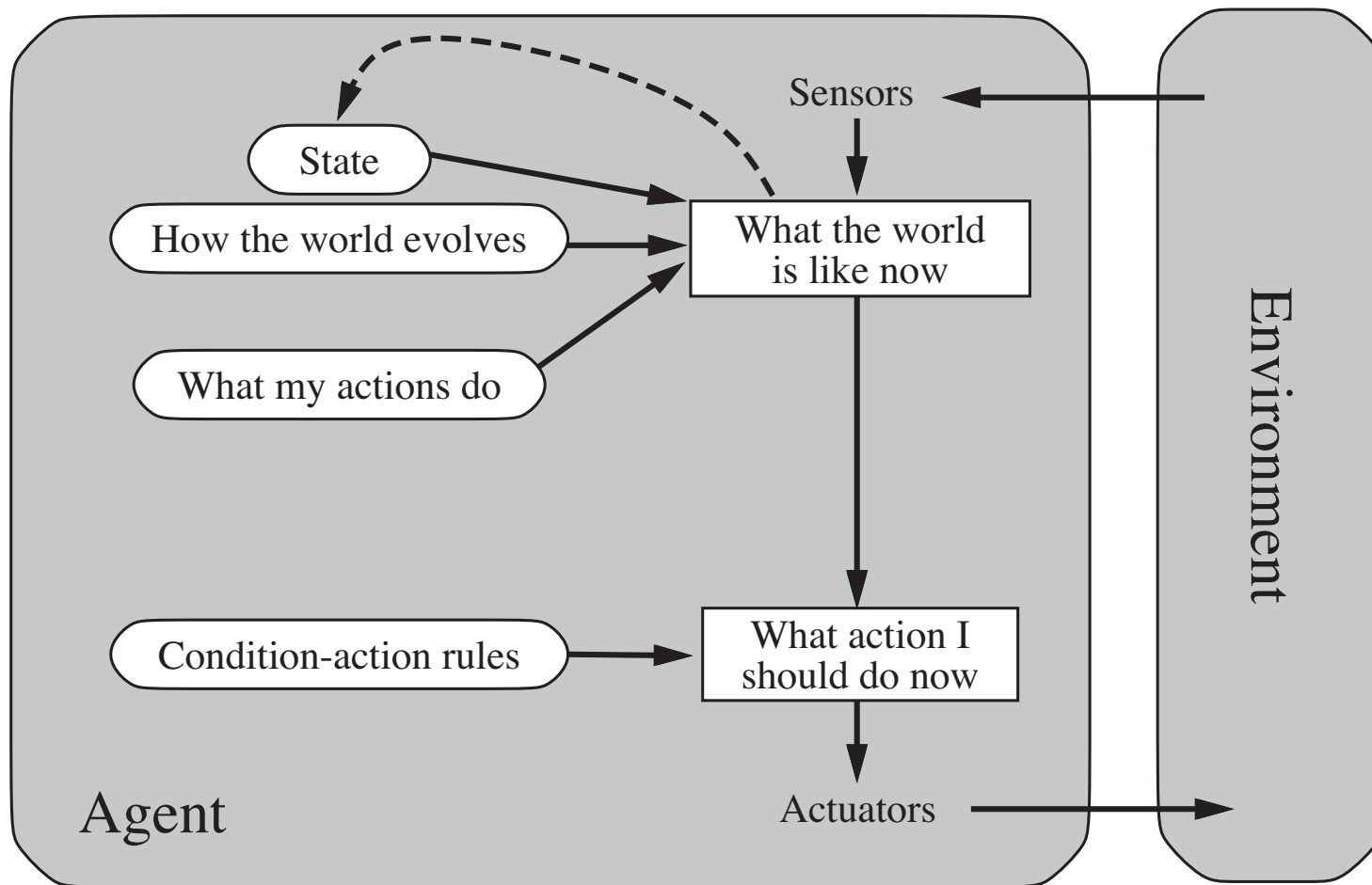
Simple Reactive Agents



Simple Reactive Agents



Model Based Agents



Symbolic Reasoning Agents

- The classical approach to building agents is to view them as a particular type of knowledge-based system, and bring all the associated (discredited?!) methodologies of such systems to bear
- This paradigm is known as *symbolic AI*
- We define a deliberative agent or agent architecture to be one that:
 - contains an explicitly represented, symbolic model of the world
 - makes decisions (for example about what actions to perform) via symbolic reasoning

Symbolic Reasoning Agents

- If we aim to build an agent in this way, there are two key problems to be solved:
 1. *The transduction problem:*
that of translating the real world into an accurate, adequate symbolic description, in time for that description to be useful...vision, speech understanding, learning
 2. *The representation/reasoning problem:*
that of how to symbolically represent information about complex real-world entities and processes, and how to get agents to reason with this information in time for the results to be useful...knowledge representation, automated reasoning, automatic planning

Deductive Reasoning Agents

- How can an agent decide what to do using theorem proving?
- Basic idea is to use logic to encode a theory stating the *best* action to perform in any given situation
- Let:
 - ρ be this theory (typically a set of rules)
 - Δ be a logical database that describes the current state of the world
 - Ac be the set of actions the agent can perform
 - $\Delta \mid_{\rho} \phi$ mean that ϕ can be proved from Δ using ρ

Deductive Reasoning Agents

/ try to find an action explicitly prescribed */*

for each $a \in Ac$ do

 if $\Delta \mid_{\rho} Do(a)$ then

 return a

 end-if

end-for

/ try to find an action not excluded */*

for each $a \in Ac$ do

 if $\Delta \not\mid_{\rho} \neg Do(a)$ then

 return a

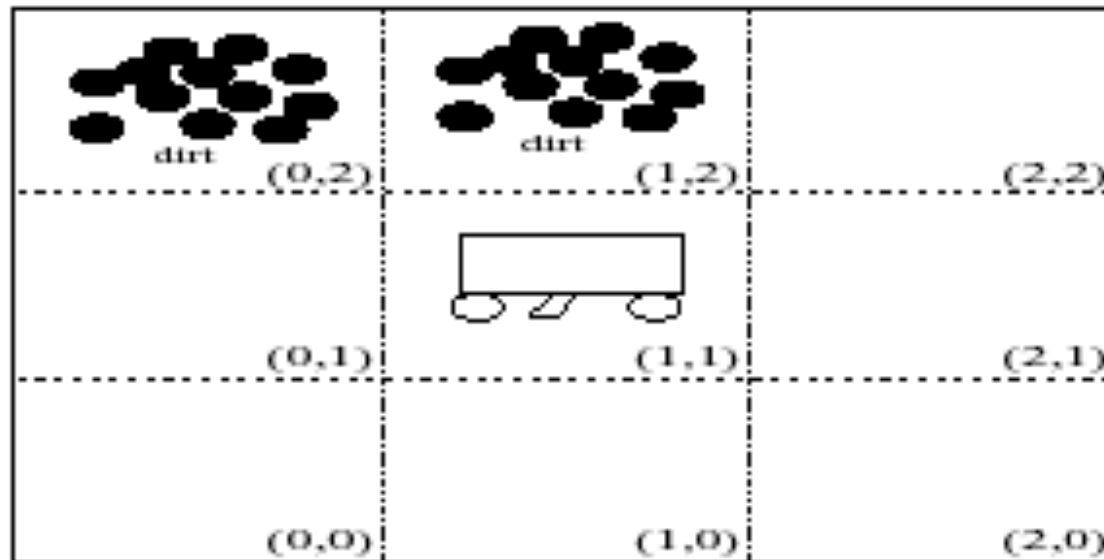
 end-if

end-for

return *null* */* no action found */*

Deductive Reasoning Agents

- An example: The Vacuum World
- Goal is for the robot to clear up all dirt



Deductive Reasoning Agents

- Use 3 *domain predicates* to solve problem:

$In(x, y)$ agent is at (x, y)

$Dirt(x, y)$ there is dirt at (x, y)

$Facing(d)$ the agent is facing direction d

- Possible actions:

$Ac = \{turn, forward, suck\}$

P.S. *turn* means “turn right”

Deductive Reasoning Agents

- Rules ρ for determining what to do:

$In(0,0) \wedge Facing(north) \wedge \neg Dirt(0,0) \longrightarrow Do(forward)$

$In(0,1) \wedge Facing(north) \wedge \neg Dirt(0,1) \longrightarrow Do(forward)$

$In(0,2) \wedge Facing(north) \wedge \neg Dirt(0,2) \longrightarrow Do(turn)$

$In(0,2) \wedge Facing(east) \longrightarrow Do(forward)$

- ...and so on!
- Using these rules (+ other obvious ones), starting at (0, 0) the robot will clear up dirt

Deductive Reasoning Agents

- Problems:
 - How to convert video camera input to *Dirt*(0, 1)?
 - decision making assumes a *static* environment: *calculative* rationality
 - decision making using first-order logic is *undecidable*!
- Even where we use *propositional* logic, decision making in the worst case means solving co-NP-complete problems
(PS: co-NP-complete = bad news!)
- Typical solutions:
 - weaken the logic
 - use symbolic, non-logical representations
 - shift the emphasis of reasoning from *run time* to *design time*

More Problems...

- The “logical approach” that was presented implies adding and removing things from a database
- That’s not pure logic
- Early attempts at creating a “planning agent” tried to use true logical deduction to solve the problem

AGENT0 and PLACA

- Much of the interest in agents from the AI community has arisen from Shoham's notion of *agent oriented programming* (AOP)
- AOP a 'new programming paradigm, based on a societal view of computation'
- The key idea that informs AOP is that of directly programming agents in terms of intentional notions like belief, commitment, and intention
- The motivation behind such a proposal is that, as we humans use the intentional stance as an *abstraction* mechanism for representing the properties of complex systems.

In the same way that we use the intentional stance to describe humans, it might be useful to use the intentional stance to program machines.

AGENT0

- Shoham suggested that a complete AOP system will have 3 components:
 - a logic for specifying agents and describing their mental states
 - an interpreted programming language for programming agents
 - an ‘agentification’ process, for converting ‘neutral applications’ (e.g., databases) into agents
- Results only reported on first two components.
- Relationship between logic and programming language is *semantics*
- We will skip over the logic(!), and consider the first AOP language, AGENT0

AGENT0

- AGENT0 is implemented as an extension to LISP
- Each agent in AGENT0 has 4 components:
 - a set of capabilities (things the agent can do)
 - a set of initial beliefs
 - a set of initial commitments (things the agent will do)
 - a set of *commitment rules*
- The key component, which determines how the agent acts, is the commitment rule set

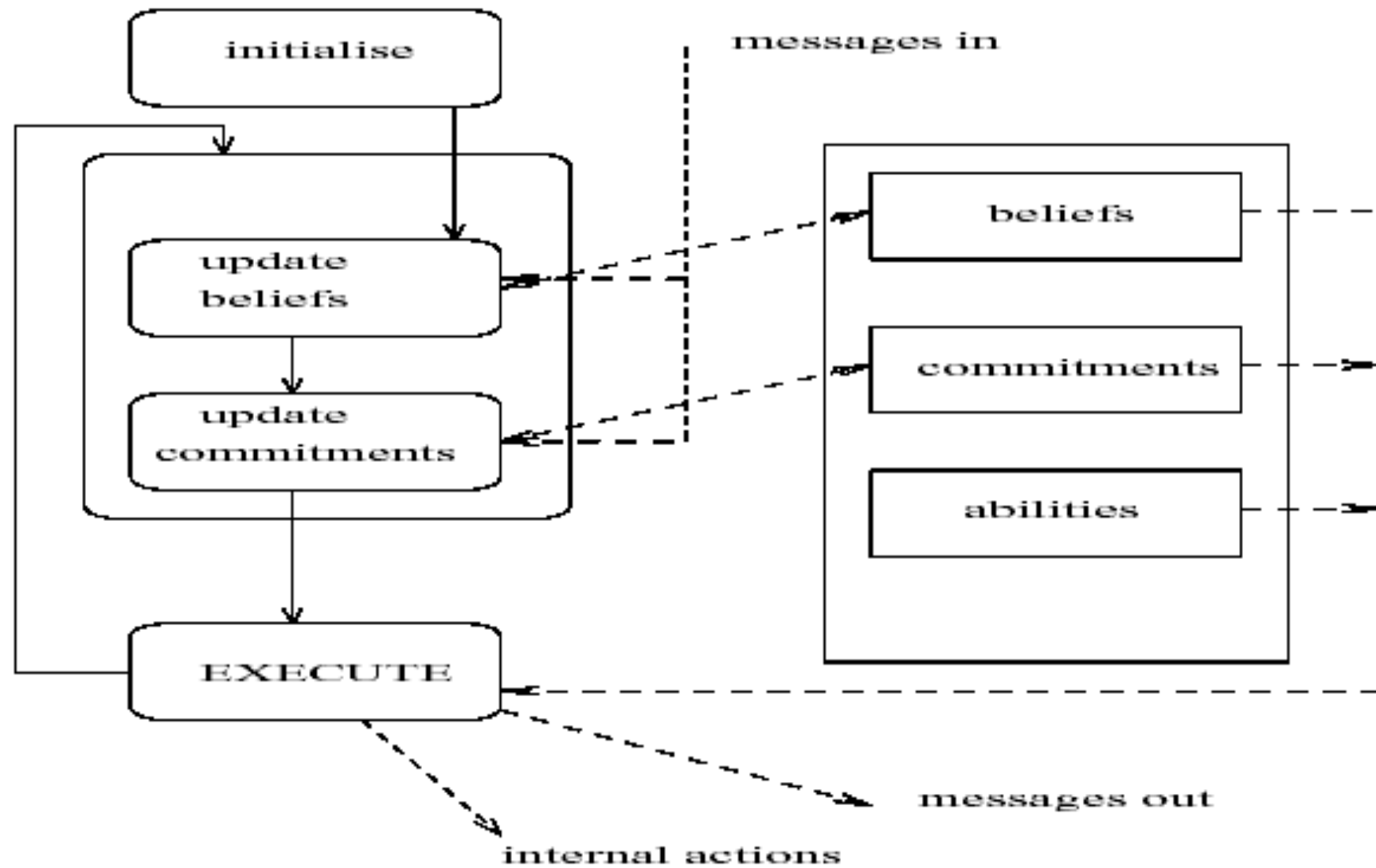
AGENT0

- Each commitment rule contains
 - *a message condition*
 - *a mental condition*
 - an action
- On each 'agent cycle'...
 - The message condition is matched against the messages the agent has received
 - The mental condition is matched against the beliefs of the agent
 - If the rule fires, then the agent becomes committed to the action (the action gets added to the agent's commitment set)

AGENT0

- Actions may be
 - *private*:
an internally executed computation, or
 - *communicative*:
sending messages
- Messages are constrained to be one of three types:
 - “requests” to commit to action
 - “unrequests” to refrain from actions
 - “informs” which pass on information

AGENT0



AGENT0

- A commitment rule:

```
COMMIT(  
    ( agent, REQUEST, DO(time, action)  
    ), ;;; msg condition  
    ( B,  
        [now, Friend agent] AND  
        CAN(self, action) AND  
        NOT [time, CMT(self, anyaction)]  
    ), ;;; mental condition  
    self,  
    DO(time, action)  
)
```

AGENT0

- This rule may be paraphrased as follows:
if I receive a message from *agent* which
requests me to do *action* at *time*, and I believe
that:
 - *agent* is currently a friend
 - I can do the action
 - At *time*, I am not committed to doing any other
actionthen commit to doing *action* at *time*

AGENT0 and PLACA

- AGENT0 provides support for multiple agents to cooperate and communicate, and provides basic provision for debugging...
- ...it is, however, a *prototype*, that was designed to illustrate some principles, rather than be a production language
- A more refined implementation was developed by Thomas, for her 1993 doctoral thesis
- Her Planning Communicating Agents (PLACA) language was intended to address one severe drawback to AGENT0: the inability of agents to plan, and communicate requests for action via high-level goals
- Agents in PLACA are programmed in much the same way as in AGENT0, in terms of *mental change* rules

AGENT0 and PLACA

- An example mental change rule:

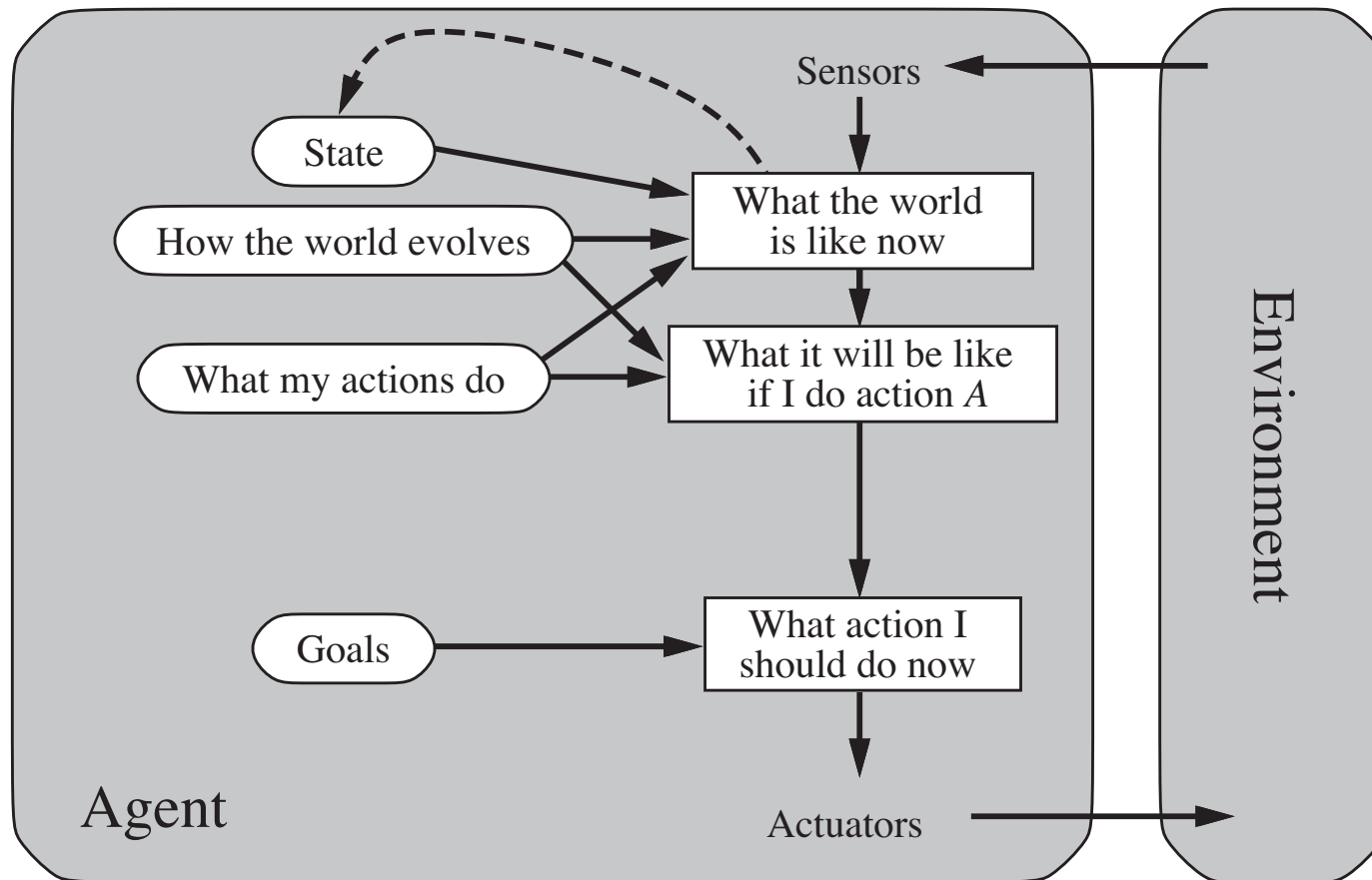
```
((self ?agent REQUEST (?t (xeroxed ?x)))  
  (AND (CAN-ACHIEVE (?t xeroxed ?x))  
        (NOT (BEL (*now* shelving)))  
        (NOT (BEL (*now* (vip ?agent)))))  
  ((ADOPT (INTEND (5pm (xeroxed ?x))))))  
  ((?agent self INFORM  
    (*now* (INTEND (5pm (xeroxed ?x)))))))
```

- Paraphrased:

if someone asks you to xerox something, and you can, and you don't believe that they're a VIP, or that you're supposed to be shelving books, then

- adopt the intention to xerox it by 5pm, and
- inform them of your newly adopted intention

Goal Based Agents



Practical Reasoning

- Practical reasoning is reasoning directed towards actions — the process of figuring out what to do:
 - “Practical reasoning is a matter of weighing conflicting considerations for and against competing options, where the relevant considerations are provided by what the agent desires/values/cares about and what the agent believes.” (Bratman)
- Practical reasoning is distinguished from *theoretical reasoning* – theoretical reasoning is directed towards beliefs

Practical Reasoning

- Human practical reasoning consists of two activities:
 - *deliberation*
deciding *what* state of affairs we want to achieve
 - *means-ends reasoning*
deciding *how* to achieve these states of affairs
- The outputs of deliberation are *intentions*

Intentions in Practical Reasoning

1. Intentions pose problems for agents, who need to determine ways of achieving them.
If I have an intention to ϕ , you would expect me to devote resources to deciding how to bring about ϕ .
2. Intentions provide a “filter” for adopting other intentions, which must not conflict.
If I have an intention to ϕ , you would not expect me to adopt an intention ψ such that ϕ and ψ are mutually exclusive.
3. Agents track the success of their intentions, and are inclined to try again if their attempts fail.
If an agent's first attempt to achieve ϕ fails, then all other things being equal, it will try an alternative plan to achieve ϕ .

Intentions in Practical Reasoning

4. Agents believe their intentions are possible.
That is, they believe there is at least some way that the intentions could be brought about.
5. Agents do not believe they will not bring about their intentions.
It would not be rational of me to adopt an intention to ϕ if I believed ϕ was not possible.
6. Under certain circumstances, agents believe they will bring about their intentions.
*It would not normally be rational of me to believe that I would bring my intentions about; intentions can fail.
Moreover, it does not make sense that if I believe ϕ is inevitable that I would adopt it as an intention.*

Intentions in Practical Reasoning

7. Agents need not intend all the expected side effects of their intentions.

If I believe $\phi \rightarrow \psi$ and I intend that ϕ , I do not necessarily intend ψ also. (Intentions are not closed under implication.)

This last problem is known as the *side effect* or *package deal* problem. I may believe that going to the dentist involves pain, and I may also intend to go to the dentist — but this does not imply that I intend to suffer pain!

Intentions in Practical Reasoning

- Notice that intentions are much stronger than mere desires:

“My desire to play basketball this afternoon is merely a potential influencer of my conduct this afternoon. It must vie with my other relevant desires [. . .] before it is settled what I will do. In contrast, once I intend to play basketball this afternoon, the matter is settled: I normally need not continue to weigh the pros and cons. When the afternoon arrives, I will normally just proceed to execute my intentions.” (Bratman, 1990)

Planning Agents

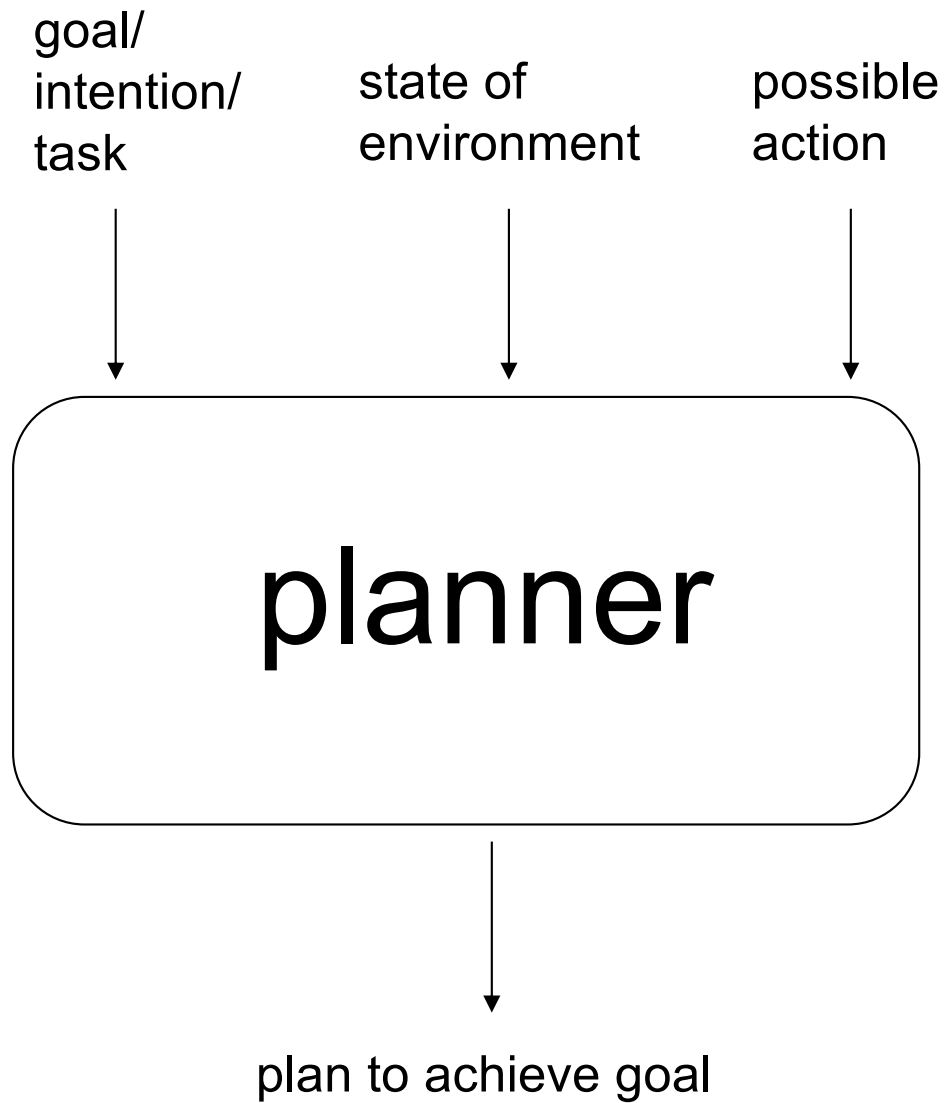
- Since the early 1970s, the AI planning community has been closely concerned with the design of artificial agents
- Planning is essentially automatic programming: the design of a course of action that will achieve some desired goal
- Within the symbolic AI community, it has long been assumed that some form of AI planning system will be a central component of any artificial agent
- Building largely on the early work of Fikes & Nilsson, many planning algorithms have been proposed, and the theory of planning has been well-developed

What is Means-End Reasoning?

- Basic idea is to give an agent:
 - representation of goal/intention to achieve
 - representation actions it can perform
 - representation of the environment

and have it generate a *plan* to achieve the goal

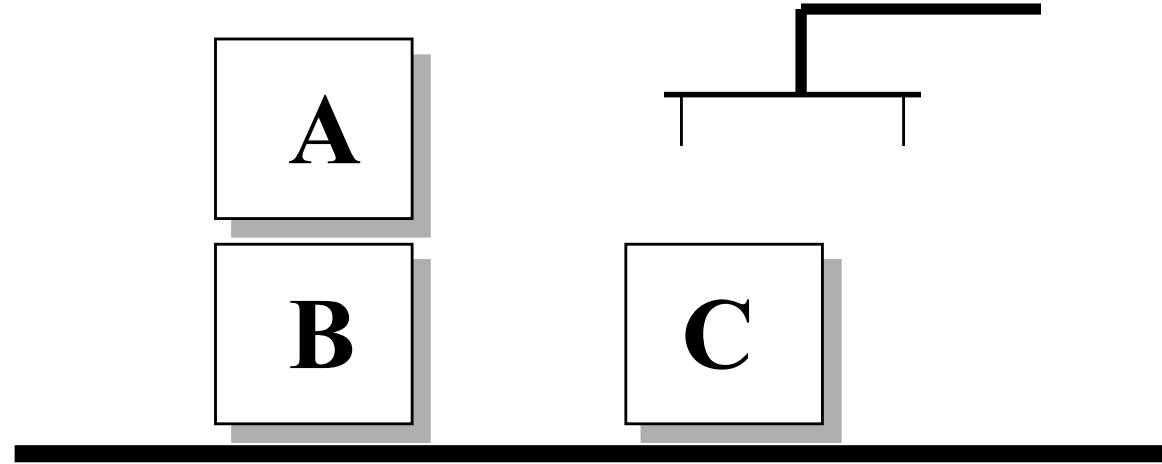
- Essentially, this is
automatic programming



Planning

- Question: How do we *represent*. . .
 - ❑ goal to be achieved
 - ❑ state of environment
 - ❑ actions available to agent
 - ❑ plan itself

The Blocks World



- We'll illustrate the techniques with reference to the *blocks world* (like last week)
- Contains a robot arm, 3 blocks (A, B, and C) of equal size, and a table-top

The Blocks World Ontology

- To represent this environment, need an *ontology*

On(x, y)

obj x on top of obj y

OnTable(x)

obj x is on the table

Clear(x)

nothing is on top of obj x

Holding(x)

arm is holding x

The Blocks World

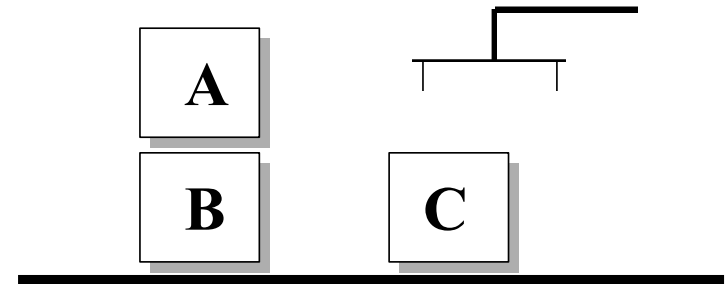
- Here is a representation of the blocks world described above:

Clear(A)

On(A, B)

OnTable(B)

OnTable(C)

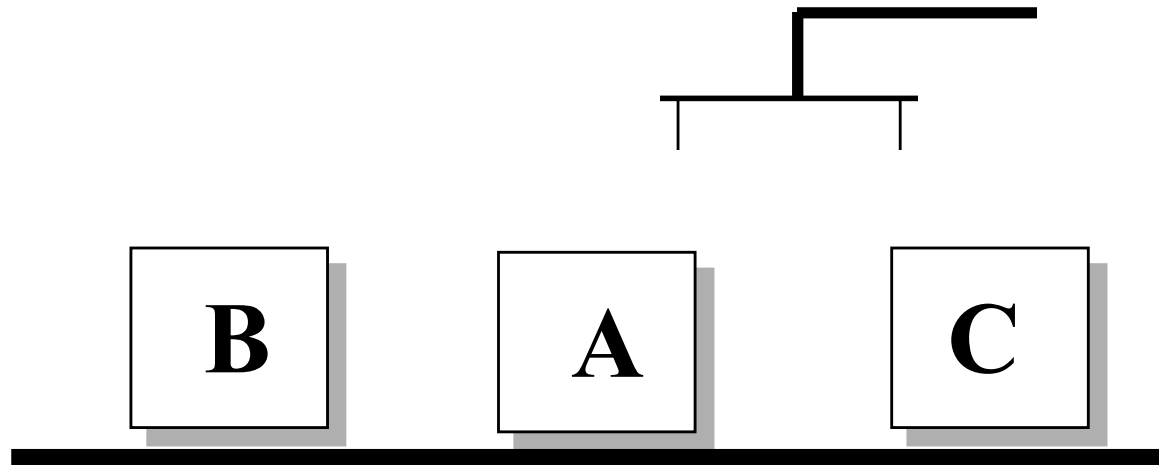


- Use the *closed world assumption*: anything not stated is assumed to be *false*

The Blocks World

- A *goal* is represented as a set of formulae
- Here is a goal:

$$OnTable(A) \wedge OnTable(B) \wedge OnTable(C)$$

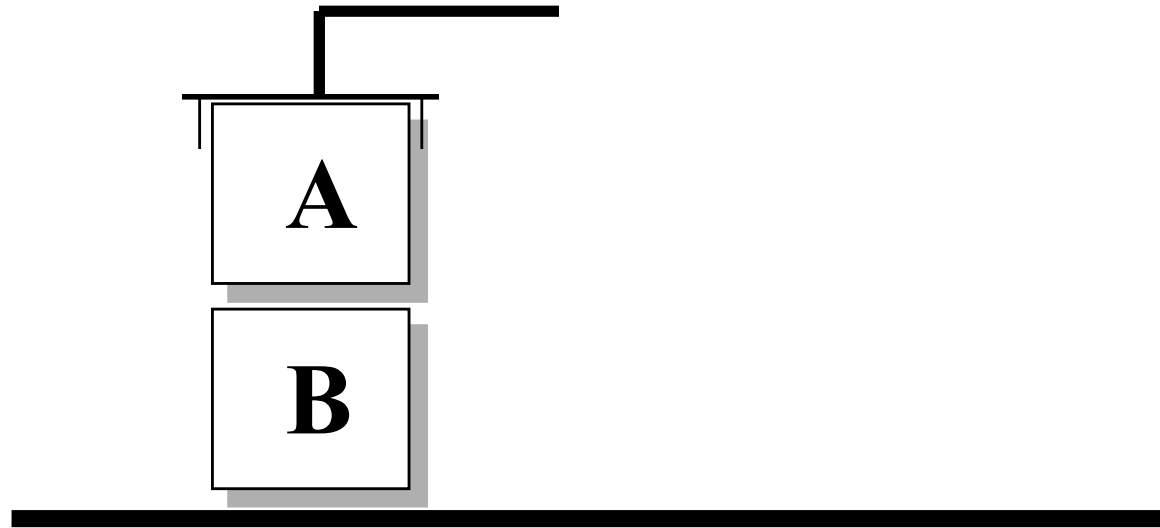


The Blocks World

- *Actions* are represented using a technique that was developed in the STRIPS planner
- Each action has:
 - a *name*
which may have arguments
 - a *pre-condition list*
list of facts which must be true for action to be executed
 - a *delete list*
list of facts that are no longer true after action is performed
 - an *add list*
list of facts made true by executing the action

Each of these may contain *variables*

The Blocks World Operators



- Example 1:

The *stack* action occurs when the robot arm places the object x it is holding on top of object y .

$Stack(x, y)$

pre $Clear(y) \wedge Holding(x)$

del $Clear(y) \wedge Holding(x)$

add $ArmEmpty \wedge On(x, y)$

The Blocks World Operators

- Example 2:

The *unstack* action occurs when the robot arm picks an object x up from on top of another object y .

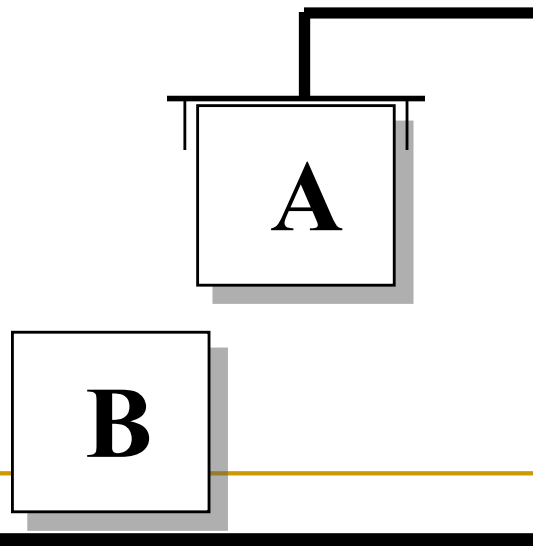
$UnStack(x, y)$

pre $On(x, y) \wedge Clear(x) \wedge ArmEmpty$

del $On(x, y) \wedge ArmEmpty$

add $Holding(x) \wedge Clear(y)$

Stack and UnStack are *inverses* of one-another.



The Blocks World Operators

- Example 3:

The *pickup* action occurs when the arm picks up an object x from the table.

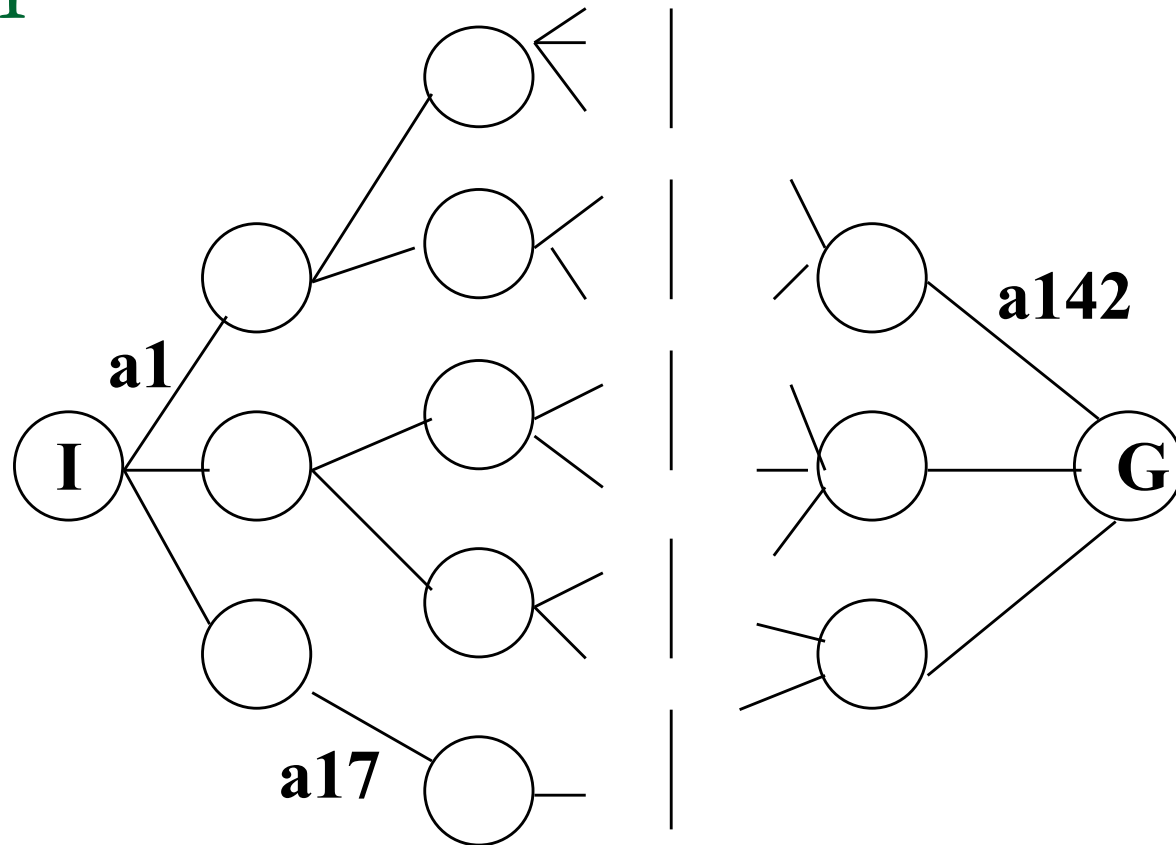
	<i>Pickup(x)</i>
pre	$\text{Clear}(x) \wedge \text{OnTable}(x) \wedge \text{ArmEmpty}$
del	$\text{OnTable}(x) \wedge \text{ArmEmpty}$
add	$\text{Holding}(x)$

- Example 4:

The *putdown* action occurs when the arm places the object x onto the table.

	<i>Putdown(x)</i>
pre	$\text{Holding}(x)$
del	$\text{Holding}(x)$
add	$\text{Clear}(x) \wedge \text{OnTable}(x) \wedge \text{ArmEmpty}$

A Plan



- What is a plan?
A sequence (list) of actions, with variables replaced by constants.

The STRIPS approach

- The original STRIPS system used a goal stack to control its search
- The system has a database and a goal stack, and it focuses attention on solving the top goal (which may involve solving subgoals, which are then pushed onto the stack, etc.)

The Basic STRIPS Idea

- Place goal on goal stack:

Goal1



- Considering top Goal1, place onto it its subgoals:

GoalS1-2

GoalS1-1

Goal1



- Then try to solve subgoal GoalS1-2, and continue...

Stack Manipulation Rules, STRIPS

If on top of goal stack:

Then do:

Compound or single goal
matching the current state description

Remove it

Compound goal *not* matching
the current state description

1. Keep original compound goal on stack
2. List the unsatisfied component goals on the stack in some *new* order

Single-literal goal not matching the
current state description

- Find rule whose instantiated
add-list includes the goal, and
1. Replace the goal with the
instantiated rule;
 2. Place the rule's instantiated
precondition formula on top of stack

Rule

1. Remove rule from stack;
2. Update database using rule;
3. Keep track of rule (for solution)

Nothing

Stop

“Underspecified” – there are decision branches here within the search tree... 4-47

Implementing Practical Reasoning Agents

- A first pass at an implementation of a practical reasoning agent:

Agent Control Loop Version 1

```
1. while true
2.     observe the world;
3.     update internal world model;
4.     deliberate about what intention to achieve next;
5.     use means-ends reasoning to get a plan for the intention;
6.     execute the plan
7. end while
```

- (We will not be concerned with stages (2) or (3))

Implementing Practical Reasoning Agents

- Problem: deliberation and means-ends reasoning processes are not instantaneous. They have a *time cost*.
- Suppose the agent starts deliberating at t_0 , begins means-ends reasoning at t_1 , and begins executing the plan at time t_2 . Time to deliberate is

$$t_{\text{deliberate}} = t_1 - t_0$$

- and time for means-ends reasoning is

$$t_{\text{me}} = t_2 - t_1$$

Implementing Practical Reasoning Agents

- Further suppose that deliberation is *optimal* in that if it selects some intention to achieve, then this is the best thing for the agent. (Maximizes expected utility.)
- So at time t_1 , the agent has selected an intention to achieve that would have been optimal *if it had been achieved at t_0* .
But unless $t_{deliberate}$ is vanishingly small, then the agent runs the risk that the intention selected is no longer optimal by the time the agent has fixed upon it.
- This is *calculative rationality*.
- Deliberation is only half of the problem: the agent still has to determine *how* to achieve the intention.

Implementing Practical Reasoning Agents

- So, this agent will have overall optimal behavior in the following circumstances:
 1. When deliberation and means-ends reasoning take a vanishingly small amount of time; or
 2. When the world is guaranteed to remain static while the agent is deliberating and performing means-ends reasoning, so that the assumptions upon which the choice of intention to achieve and plan to achieve the intention remain valid until the agent has completed deliberation and means-ends reasoning; or
 3. When an intention that is optimal when achieved at time t_0 (the time at which the world is observed) is guaranteed to remain optimal until time t_2 (the time at which the agent has found a course of action to achieve the intention).

Implementing Practical Reasoning Agents

- Let's make the algorithm more formal:

```
Agent Control Loop Version 2
1.   $B := B_0$ ; /* initial beliefs */
2.  while true do
3.      get next percept  $\rho$ ;
4.       $B := brf(B, \rho)$ ;
5.       $I := deliberate(B)$ ;
6.       $\pi := plan(B, I)$ ;
7.      execute( $\pi$ )
8.  end while
```

Deliberation

- How does an agent deliberate?
 - begin by trying to understand what the *options* available to you are
 - *choose between them*, and *commit* to some
- Chosen options are then intentions

Deliberation

- The *deliberate* function can be decomposed into two distinct functional components:
 - *option generation*
in which the agent generates a set of possible alternatives;
Represent option generation via a function, *options*, which takes the agent's current beliefs and current intentions, and from them determines a set of options (= *desires*)
 - *filtering*
in which the agent chooses between competing alternatives, and commits to achieving them.
In order to select between competing options, an agent uses a *filter* function.

Deliberation

Agent Control Loop Version 3

```
1.  
2.    $B := B_0;$   
3.    $I := I_0;$   
4.   while true do  
5.       get next percept  $\rho;$   
6.        $B := brf(B, \rho);$   
7.        $D := options(B, I);$   
8.        $I := filter(B, D, I);$   
9.        $\pi := plan(B, I);$   
10.       $execute(\pi)$   
11.  end while
```

Commitment Strategies

- The following *commitment strategies* are commonly discussed in the literature of rational agents:
 - *Blind commitment*
A blindly committed agent will continue to maintain an intention until it believes the intention has actually been achieved. Blind commitment is also sometimes referred to as *fanatical* commitment.
 - *Single-minded commitment*
A single-minded agent will continue to maintain an intention until it believes that either the intention has been achieved, or else that it is no longer possible to achieve the intention.
 - *Open-minded commitment*
An open-minded agent will maintain an intention as long as it is still believed possible.

Commitment Strategies

- An agent has commitment both to *ends* (i.e., the wishes to bring about), and *means* (i.e., the mechanism via which the agent wishes to achieve the state of affairs)
- Currently, our agent control loop is overcommitted, both to means and ends
Modification: *replan* if ever a plan goes wrong

Agent Control Loop Version 4

```
1.
2.   $B := B_0;$ 
3.   $I := I_0;$ 
4.  while true do
5.      get next percept  $\rho;$ 
6.       $B := brf(B, \rho);$ 
7.       $D := options(B, I);$ 
8.       $I := filter(B, D, I);$ 
9.       $\pi := plan(B, I);$ 
10.     while not empty( $\pi$ ) do
11.          $\alpha := hd(\pi);$ 
12.         execute( $\alpha$ );
13.          $\pi := tail(\pi);$ 
14.         get next percept  $\rho;$ 
15.          $B := brf(B, \rho);$ 
16.         if not sound( $\pi, I, B$ ) then
17.              $\pi := plan(B, I)$ 
18.         end-if
19.     end-while
20. end-while
```

Commitment Strategies

- Still overcommitted to intentions: Never stops to consider whether or not its intentions are appropriate
- Modification: stop to determine whether intentions have succeeded or whether they are impossible:
(*Single-minded commitment*)

Agent Control Loop Version 5

```
2.   $B := B_0;$ 
3.   $I := I_0;$ 
4.  while true do
5.      get next percept  $\rho$ ;
6.       $B := brf(B, \rho);$ 
7.       $D := options(B, I);$ 
8.       $I := filter(B, D, I);$ 
9.       $\pi := plan(B, I);$ 
10.     while not  $empty(\pi)$ 
           or  $succeeded(I, B)$ 
           or  $impossible(I, B)$  do
11.          $\alpha := hd(\pi);$ 
12.          $execute(\alpha);$ 
13.          $\pi := tail(\pi);$ 
14.         get next percept  $\rho$ ;
15.          $B := brf(B, \rho);$ 
16.         if not  $sound(\pi, I, B)$  then
17.              $\pi := plan(B, I)$ 
18.         end-if
19.     end-while
20. end-while
```

Intention Reconsideration

- Our agent gets to reconsider its intentions once every time around the outer control loop, i.e., when:
 - ❑ it has completely executed a plan to achieve its current intentions; or
 - ❑ it believes it has achieved its current intentions; or
 - ❑ it believes its current intentions are no longer possible.
 - This is limited in the way that it permits an agent to *reconsider* its intentions
 - Modification: Reconsider intentions after executing every action
-

Agent Control Loop Version 6

```
1.
2.   $B := B_0$ ;
3.   $I := I_0$ ;
4.  while true do
5.      get next percept  $\rho$ ;
6.       $B := brf(B, \rho)$ ;
7.       $D := options(B, I)$ ;
8.       $I := filter(B, D, I)$ ;
9.       $\pi := plan(B, I)$ ;
10.     while not ( $empty(\pi)$ 
                  or  $succeeded(I, B)$ 
                  or  $impossible(I, B)$ ) do
11.          $\alpha := hd(\pi)$ ;
12.          $execute(\alpha)$ ;
13.          $\pi := tail(\pi)$ ;
14.         get next percept  $\rho$ ;
15.          $B := brf(B, \rho)$ ;
16.          $D := options(B, I)$ ;
17.          $I := filter(B, D, I)$ ;
18.         if not  $sound(\pi, I, B)$  then
19.              $\pi := plan(B, I)$ 
20.         end-if
21.     end-while
22. end-while
```

Intention Reconsideration

- But intention reconsideration is *costly*!
A dilemma:
 - an agent that does not stop to reconsider its intentions sufficiently often will continue attempting to achieve its intentions even after it is clear that they cannot be achieved, or that there is no longer any reason for achieving them
 - an agent that *constantly* reconsiders its intentions may spend insufficient time actually working to achieve them, and hence runs the risk of never actually achieving them
- Solution: incorporate an explicit *meta-level control* component, that decides whether or not to reconsider

Agent Control Loop Version 7

```
1.
2.   $B := B_0;$ 
3.   $I := I_0;$ 
4.  while true do
5.      get next percept  $\rho$ ;
6.       $B := brf(B, \rho);$ 
7.       $D := options(B, I);$ 
8.       $I := filter(B, D, I);$ 
9.       $\pi := plan(B, I);$ 
10.     while not (empty( $\pi$ )
                  or succeeded( $I, B$ )
                  or impossible( $I, B$ )) do
11.          $\alpha := hd(\pi);$ 
12.         execute( $\alpha$ );
13.          $\pi := tail(\pi);$ 
14.         get next percept  $\rho$ ;
15.          $B := brf(B, \rho);$ 
16.         if reconsider( $I, B$ ) then
17.              $D := options(B, I);$ 
18.              $I := filter(B, D, I);$ 
19.         end-if
20.         if not sound( $\pi, I, B$ ) then
21.              $\pi := plan(B, I)$ 
22.         end-if
23.     end-while
24. end-while
```

Possible Interactions

- The possible interactions between meta-level control and deliberation are:

Situation number	Chose to deliberate?	Changed intentions?	Would have changed intentions?	<i>reconsider(...)</i> optimal?
1	No	—	No	Yes
2	No	—	Yes	No
3	Yes	No	—	No
4	Yes	Yes	—	Yes

Intention Reconsideration

- In situation (1), the agent did not choose to deliberate, and as consequence, did not choose to change intentions. Moreover, if it *had* chosen to deliberate, it would not have changed intentions. In this situation, the *reconsider(...)* function is behaving optimally.
- In situation (2), the agent did not choose to deliberate, but if it had done so, it *would* have changed intentions. In this situation, the *reconsider(...)* function is not behaving optimally.
- In situation (3), the agent chose to deliberate, but did not change intentions. In this situation, the *reconsider(...)* function is not behaving optimally.
- In situation (4), the agent chose to deliberate, and did change intentions. In this situation, the *reconsider(...)* function is behaving optimally.
- An important assumption: cost of *reconsider(...)* is *much* less than the cost of the deliberation process itself.

Optimal Intention Reconsideration

- Kinny and Georgeff's experimentally investigated effectiveness of intention reconsideration strategies
- Two different types of reconsideration strategy were used:
 - *bold* agents
never pause to reconsider intentions, and
 - *cautious* agents
stop to reconsider after every action
- *Dynamism* in the environment is represented by the *rate of world change*, γ

Optimal Intention Reconsideration

■ Results (not surprising):

- If γ is low (i.e., the environment does not change quickly), then bold agents do well compared to cautious ones. This is because cautious ones waste time reconsidering their commitments while bold agents are busy working towards — and achieving — their intentions.
- If γ is high (i.e., the environment changes frequently), then cautious agents tend to outperform bold agents. This is because they are able to recognize when intentions are doomed, and also to take advantage of serendipitous situations and new opportunities when they arise.

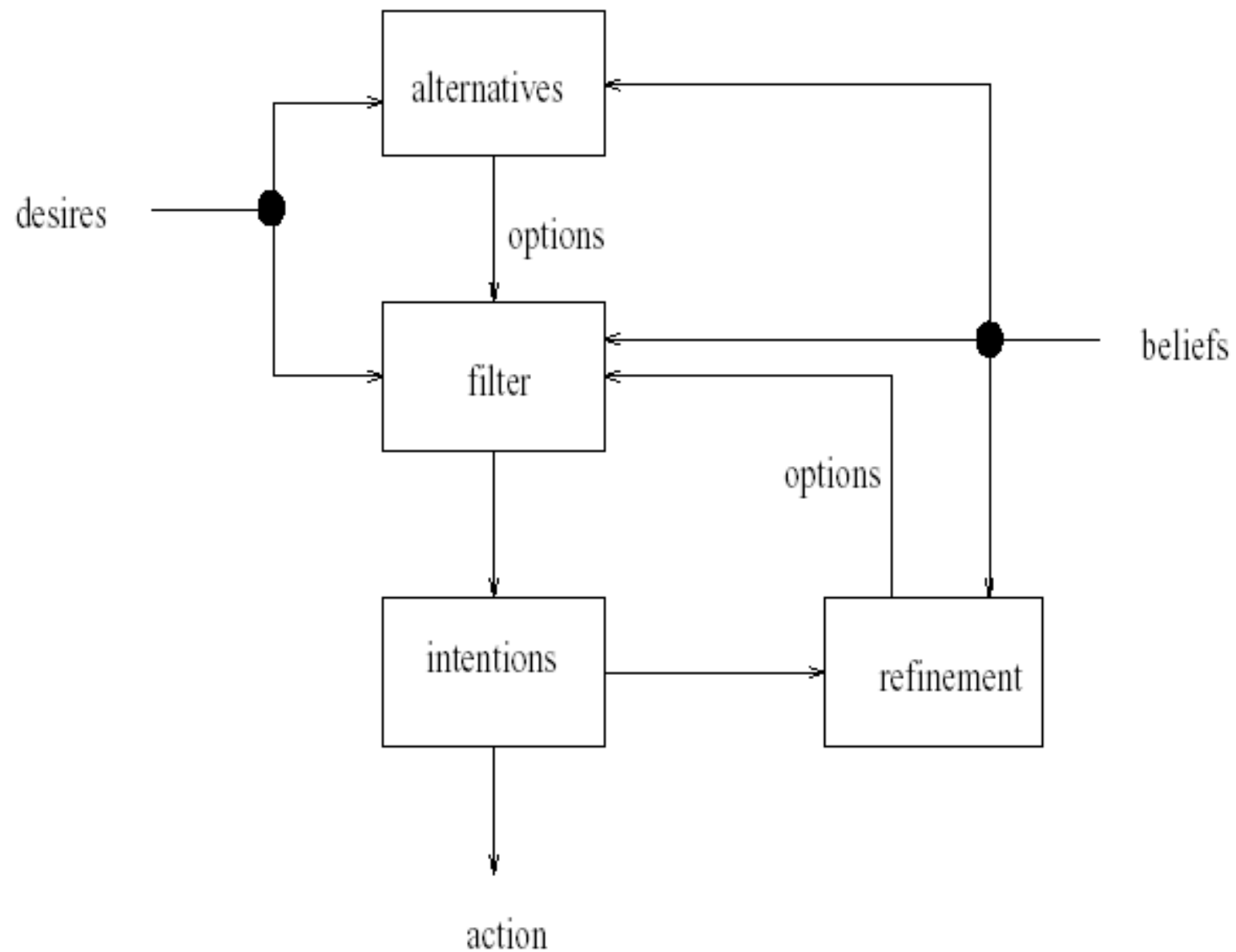
Implemented BDI Agents: IRMA

- IRMA – Intelligent Resource-bounded Machine Architecture – Bratman, Israel, Pollack
- IRMA has four key symbolic data structures:
 - a *plan library*
 - explicit representations of
 - *beliefs*: information available to the agent — may be represented symbolically, but may be simple variables
 - *desires*: those things the agent would *like* to make true — think of desires as *tasks* that the agent has been allocated; in humans, not necessarily logically consistent, but our agents will be! (goals)
 - *intentions*: desires that the agent has *chosen* and *committed to*

IRMA

- Additionally, the architecture has:
 - a *reasoner* for reasoning about the world; an inference engine
 - a *means-ends analyzer* determines which plans might be used to achieve intentions
 - an *opportunity analyzer* monitors the environment, and as a result of changes, generates new options
 - a *filtering process* determines which options are compatible with current intentions
 - a *deliberation process* responsible for deciding upon the 'best' intentions to adopt

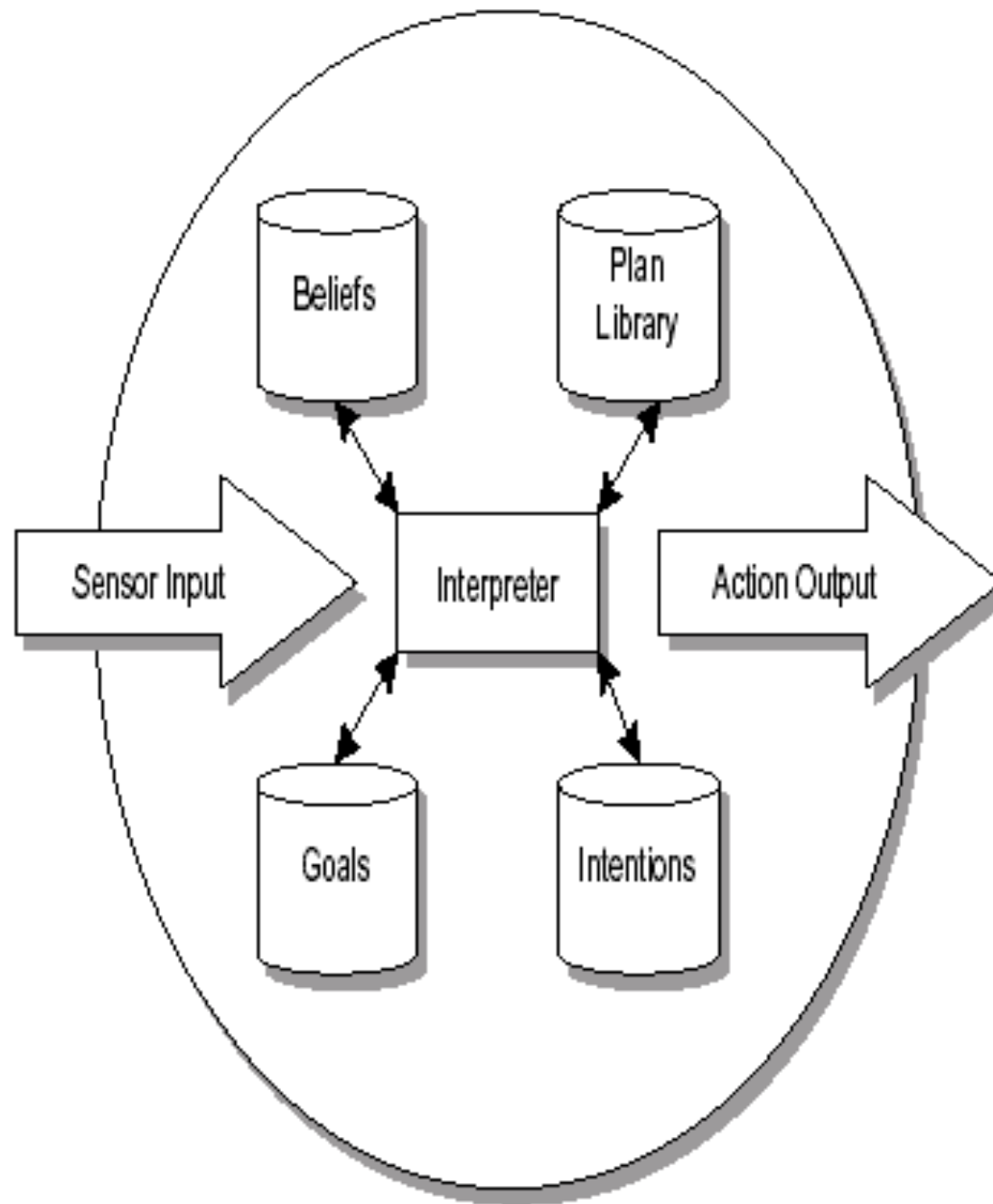
IRMA



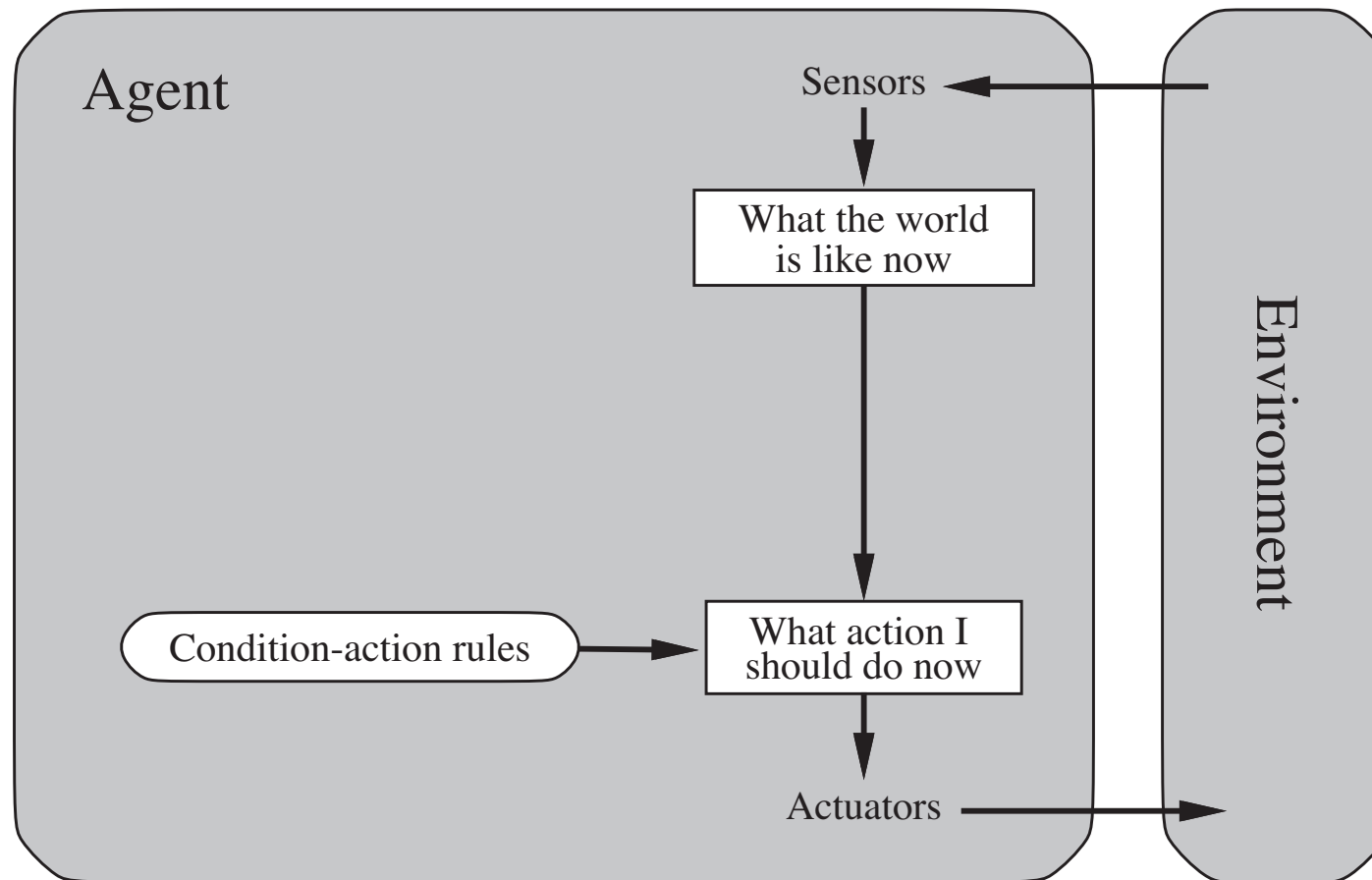
Implemented BDI Agents: PRS

- Another BDI-based agent architecture: the PRS – Procedural Reasoning System (Georgeff, Lansky)
- In the PRS, each agent is equipped with a *plan library*, representing that agent's *procedural knowledge*: knowledge about the mechanisms that can be used by the agent in order to realize its intentions
- The options available to an agent are directly determined by the plans an agent has: an agent with no plans has no options
- In addition, PRS agents have explicit representations of beliefs, desires, and intentions, as above

PRS



Simple Reactive Agents



Brooks – behavior languages

- Brooks has put forward three theses:
 1. Intelligent behavior can be generated *without* explicit representations of the kind that symbolic AI proposes
 2. Intelligent behavior can be generated *without* explicit abstract reasoning of the kind that symbolic AI proposes
 3. Intelligence is an *emergent* property of certain complex systems

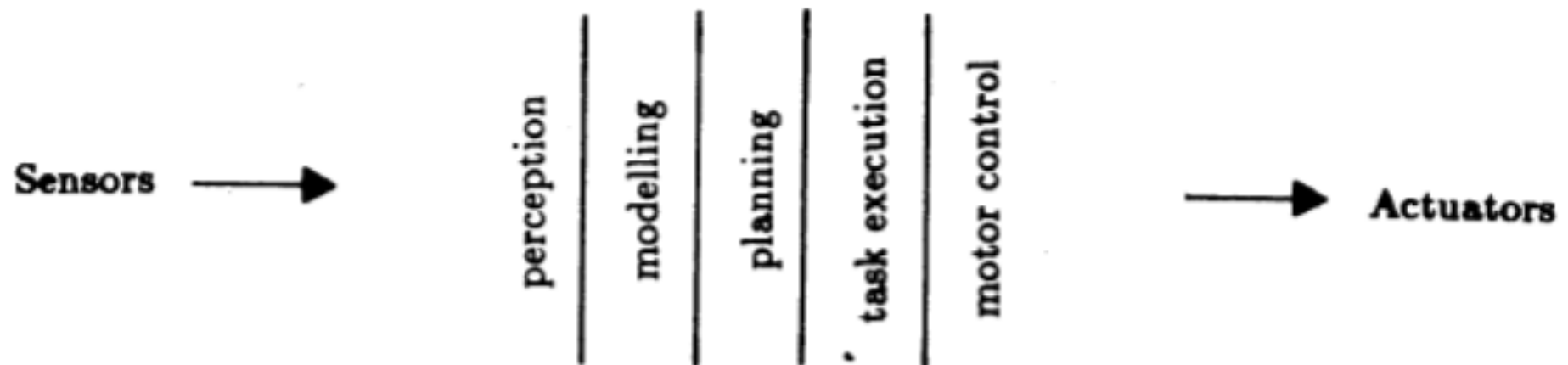
Brooks – behavior languages

- He identifies two key ideas that have informed his research:
 1. Situatedness and embodiment: ‘Real’ intelligence is situated in the world, not in disembodied systems such as theorem provers or expert systems
 2. Intelligence and emergence: ‘Intelligent’ behavior arises as a result of an agent’s interaction with its environment. Also, intelligence is ‘in the eye of the beholder’; it is not an innate, isolated property

Brooks – behavior languages

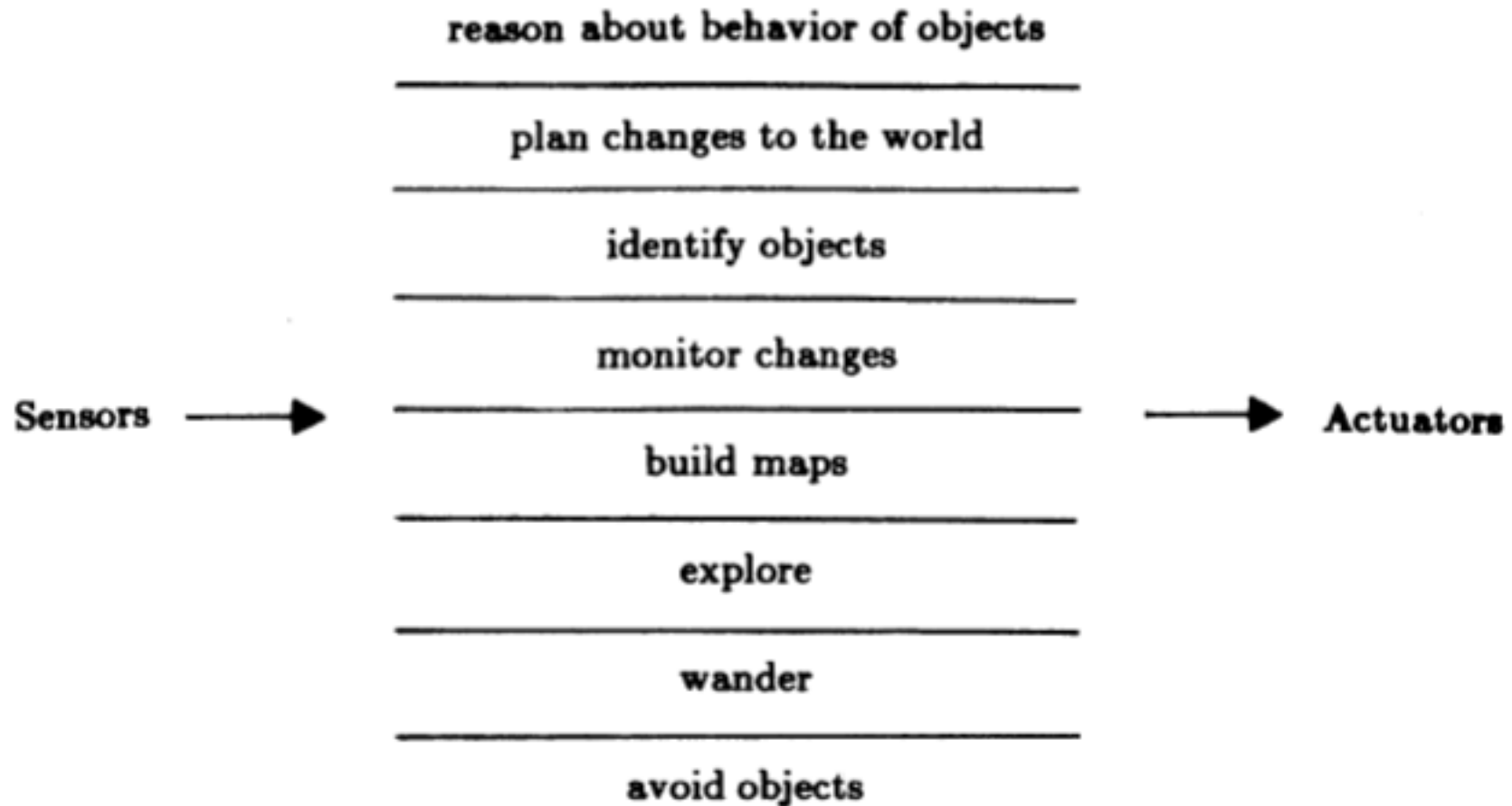
- To illustrate his ideas, Brooks built some based on his *subsumption architecture*
- A subsumption architecture is a hierarchy of task-accomplishing *behaviors*
- Each behavior is a rather simple rule-like structure
- Each behavior ‘competes’ with others to exercise control over the agent
- Lower layers represent more primitive kinds of behavior (such as avoiding obstacles), and have precedence over layers further up the hierarchy
- The resulting systems are, in terms of the amount of computation they do, *extremely* simple
- Some of the robots do tasks that would be impressive if they were accomplished by symbolic AI systems

A Traditional Decomposition of a Mobile Robot Control System into Functional Modules



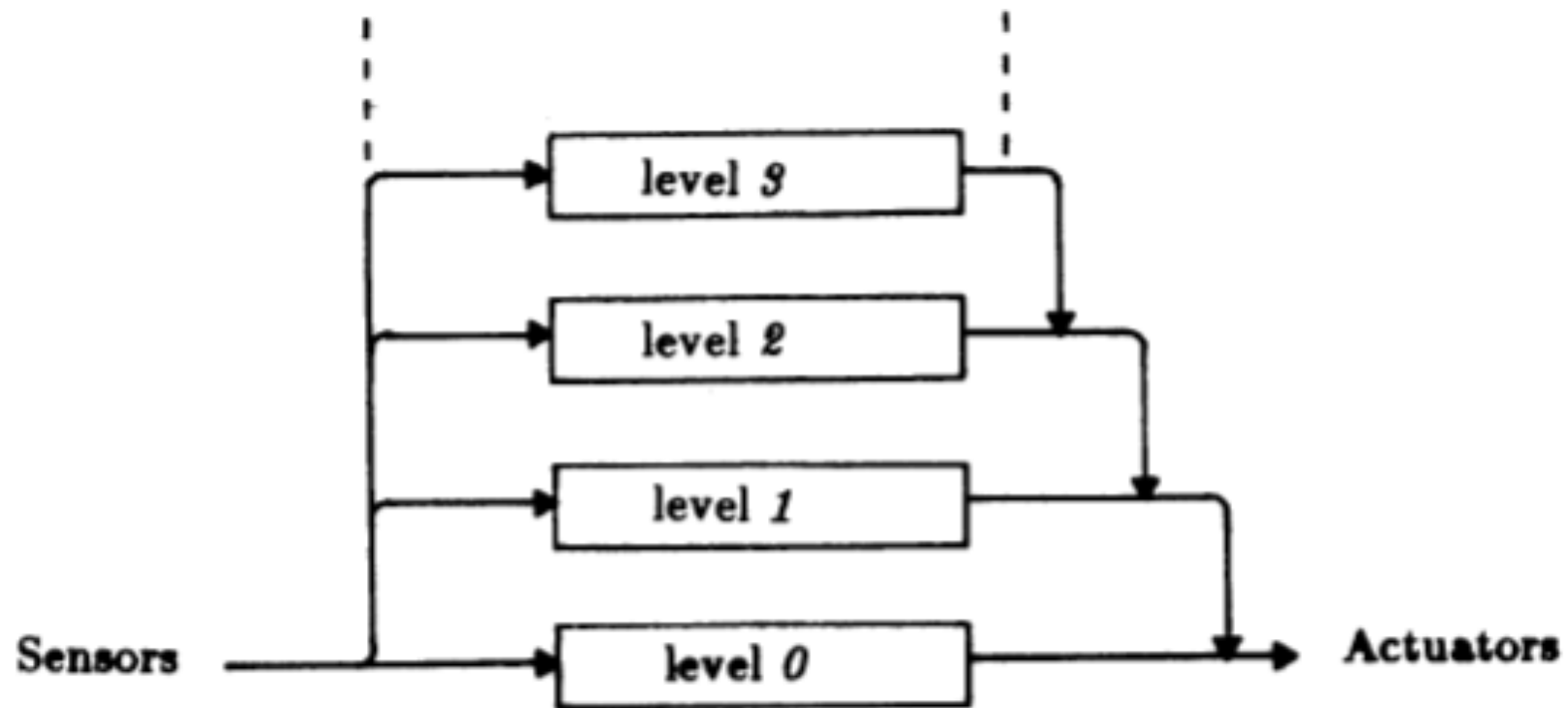
From Brooks, “A Robust Layered Control System for a Mobile Robot”, 1985

A Decomposition of a Mobile Robot Control System Based on Task Achieving Behaviors



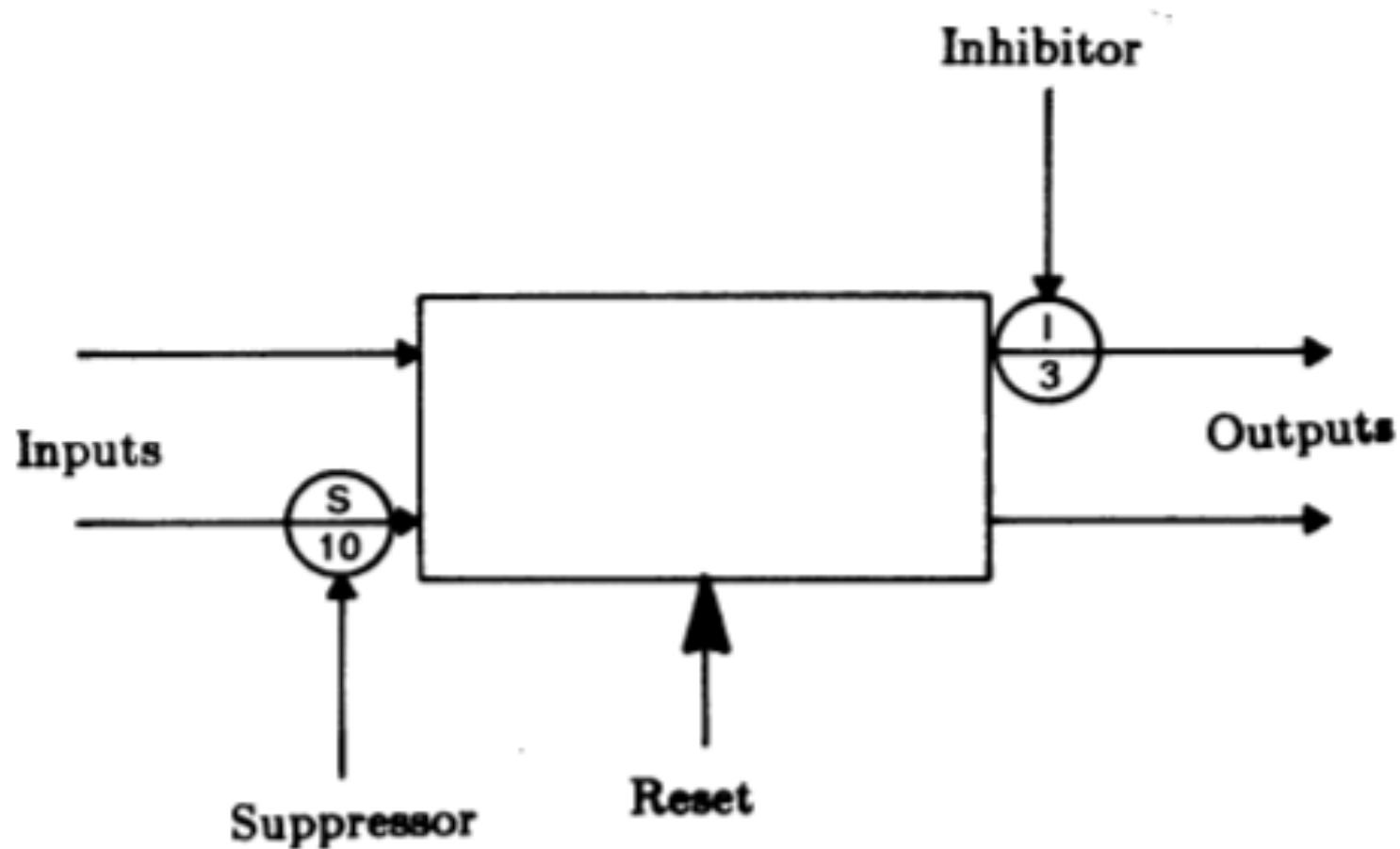
From Brooks, "A Robust Layered Control System for a Mobile Robot", 1985

Layered Control in the Subsumption Architecture



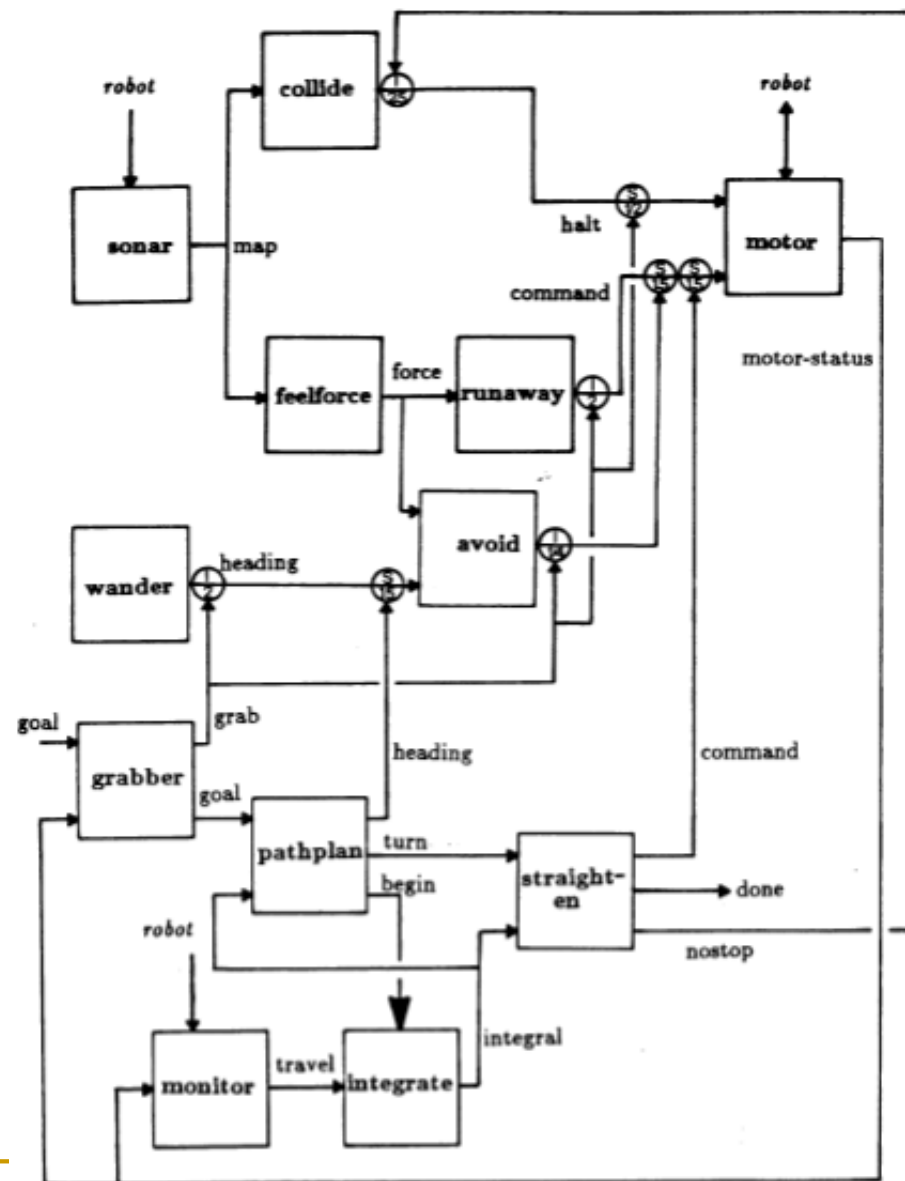
From Brooks, "A Robust Layered Control System for a Mobile Robot", 1985

Schematic of a Module



From Brooks, "A Robust Layered Control System for a Mobile Robot", 1985

Levels 0, 1, and 2 Control Systems



Steels' Mars Explorer

- Steels' Mars explorer system, using the subsumption architecture, achieves near-optimal cooperative performance in simulated 'rock gathering on Mars' domain:
The objective is to explore a distant planet, and in particular, to collect sample of a precious rock. The location of the samples is not known in advance, but it is known that they tend to be clustered.

Steels' Mars Explorer Rules

- For individual (non-cooperative) agents, the lowest-level behavior, (and hence the behavior with the highest “priority”) is obstacle avoidance:
if detect an obstacle then change direction (1)
- Any samples carried by agents are dropped back at the mother-ship:
*if carrying samples and at the base
then drop samples* (2)
- Agents carrying samples will return to the mother-ship:
*if carrying samples and not at the base
then travel up gradient* (3)

Steels' Mars Explorer Rules

- Agents will collect samples they find:
if detect a sample then pick sample up (4)
- An agent with “nothing better to do” will explore randomly:
if true then move randomly (5)

Advantages of Reactive Agents

- Simplicity
- Economy
- Computational tractability
- Robustness against failure
- Elegance

Limitations of Reactive Agents

- Agents without environment models must have sufficient information available from local environment
- If decisions are based on *local* environment, how does it take into account *non-local* information (i.e., it has a “short-term” view)
- Since behavior emerges from component interactions plus environment, it is hard to see how to *engineer* specific agents (no principled methodology exists)
- It is hard to engineer agents with large numbers of behaviors (dynamics of interactions become too complex to understand)

Hybrid Architectures

- Many researchers have argued that neither a completely deliberative nor completely reactive approach is suitable for building agents
- They have suggested using *hybrid* systems, which attempt to marry classical and alternative approaches
- An obvious approach is to build an agent out of two (or more) subsystems:
 - a *deliberative* one, containing a symbolic world model, which develops plans and makes decisions in the way proposed by symbolic AI
 - a *reactive* one, which is capable of reacting to events without complex reasoning

Hybrid Architectures

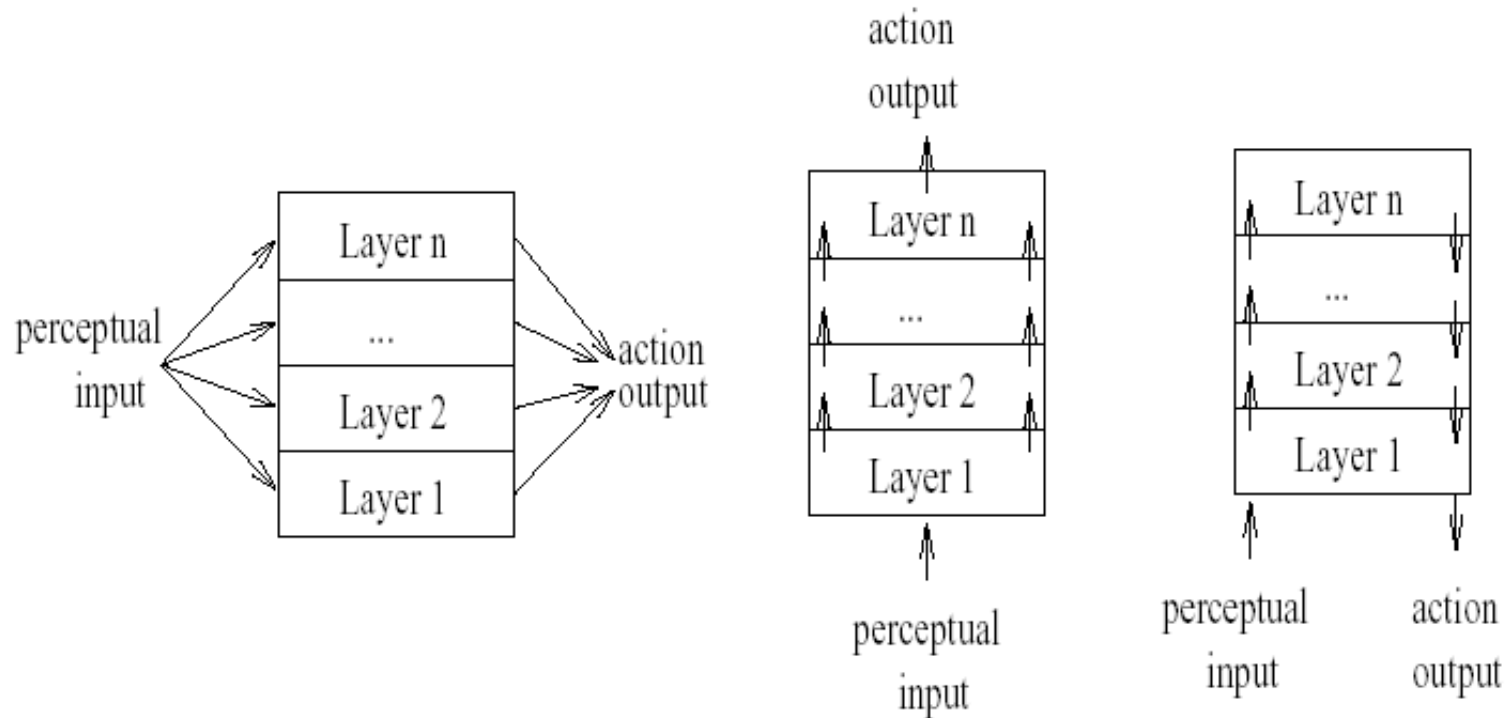
- Often, the reactive component is given some kind of precedence over the deliberative one
- This kind of structuring leads naturally to the idea of a *layered* architecture, of which TOURINGMACHINES and INTERRAP are examples
- In such an architecture, an agent's control subsystems are arranged into a hierarchy, with higher layers dealing with information at increasing levels of abstraction

Hybrid Architectures

- A key problem in such architectures is what kind of control framework to embed the agent's subsystems in, to manage the interactions between the various layers
- *Horizontal layering*
Layers are each directly connected to the sensory input and action output.
In effect, each layer itself acts like an agent, producing suggestions as to what action to perform.
- *Vertical layering*
Sensory input and action output are each dealt with by at most one layer each

Hybrid Architectures

m possible actions suggested by each layer, n layers



(a) Horizontal layering

(b) Vertical layering
(One pass control)

(c) Vertical layering
(Two pass control)

m^n interactions

$m^2(n-1)$ interactions

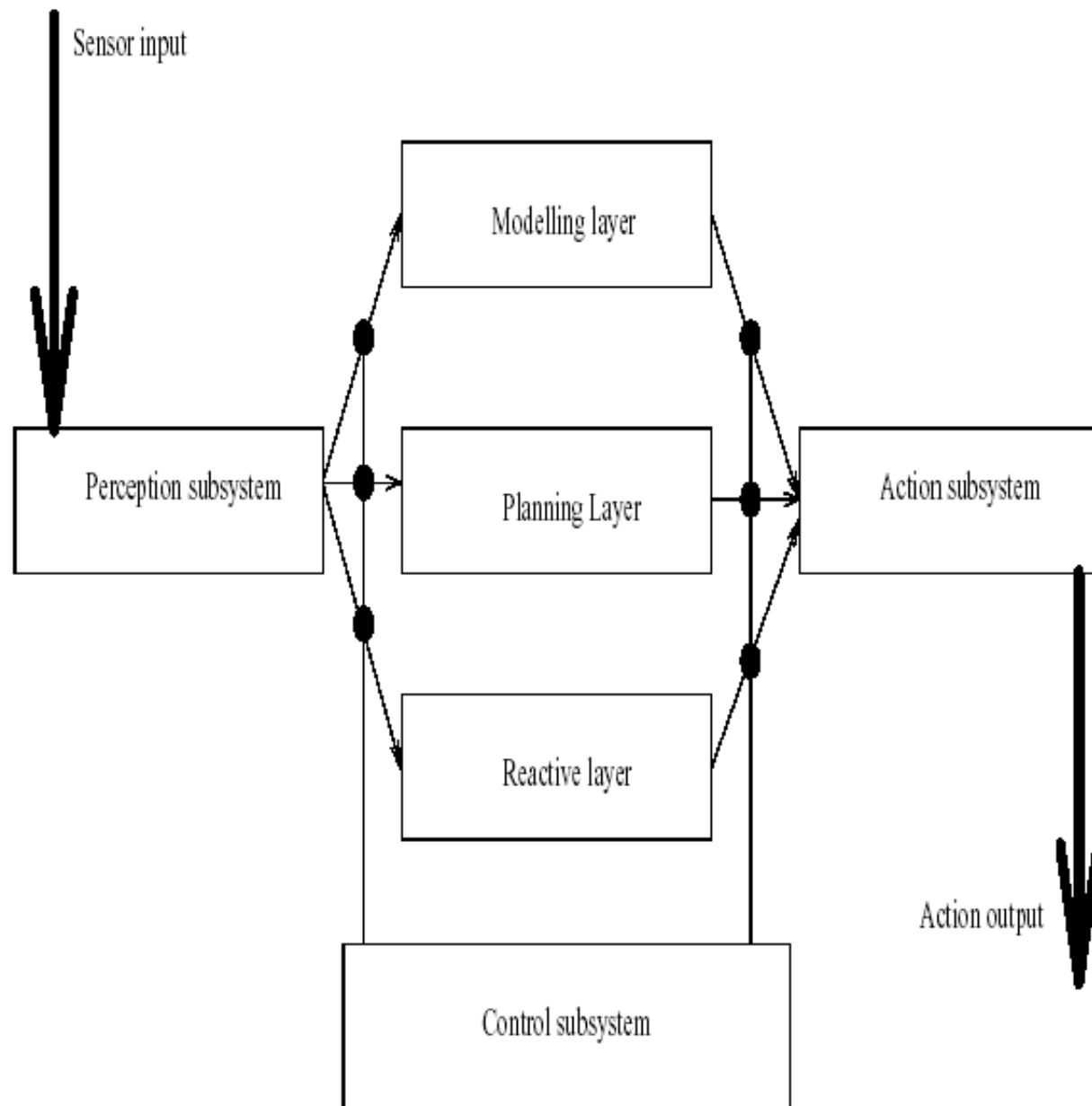
Introduces bottleneck
in central control system

Not fault tolerant to
layer failure

Ferguson – TOURINGMACHINES

- The TOURINGMACHINES architecture consists of *perception* and *action* subsystems, which interface directly with the agent's environment, and three *control layers*, embedded in a *control framework*, which mediates between the layers

Ferguson – TOURINGMACHINES



Ferguson – TOURINGMACHINES

- The *reactive layer* is implemented as a set of situation-action rules, *a la* subsumption architecture

Example:

rule-1: kerb-avoidance

if

is-in-front(Kerb, Observer) and

speed(Observer) > 0 and

separation(Kerb, Observer) < KerbThreshold

then

change-orientation(KerbAvoidanceAngle)

- The *planning layer* constructs plans and selects actions to execute in order to achieve the agent's goals

Ferguson – TOURINGMACHINES

- The *modeling layer* contains symbolic representations of the ‘cognitive state’ of other entities in the agent’s environment
- The three layers communicate with each other and are embedded in a control framework, which use *control rules*

Example:

sensor-rule-1:

if

entity(obstacle-6) in perception-buffer

then

remove-sensory-record(layer-R, entity(obstacle-6))

Müller –InteRRaP

- Vertically layered, two-pass architecture

